05/07/2026

Investigating Deployment and Coexistence Challenges for Low-Latency Congestion Control

Fatih Berkay Sarpkaya

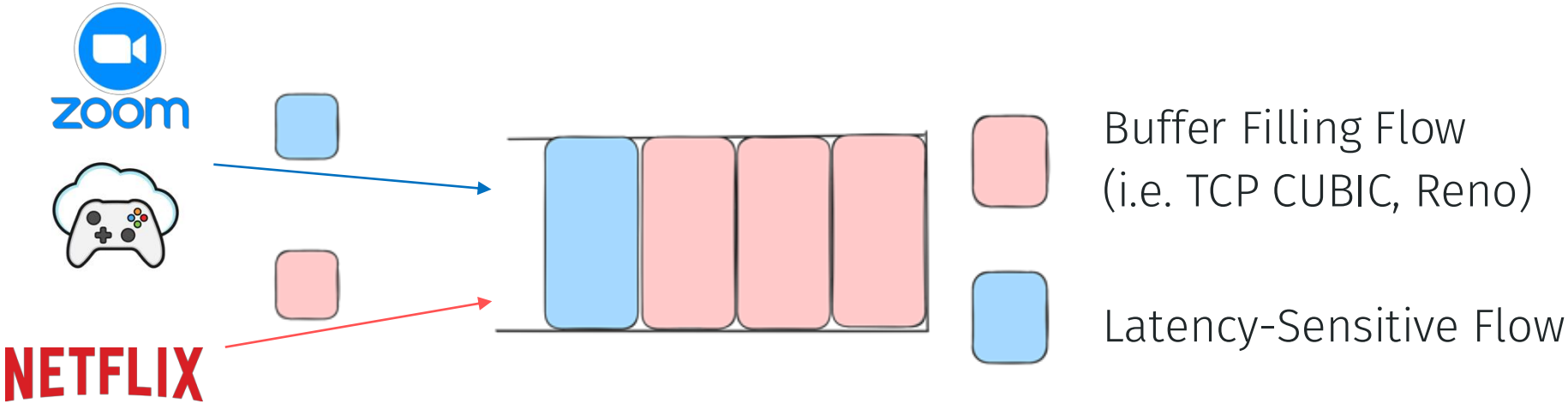
Area Exam Presentation

NYU TANDON SCHOOL OF ENGINEERING

Low-Latency Protocols Coexistence Problem

Latency Problem

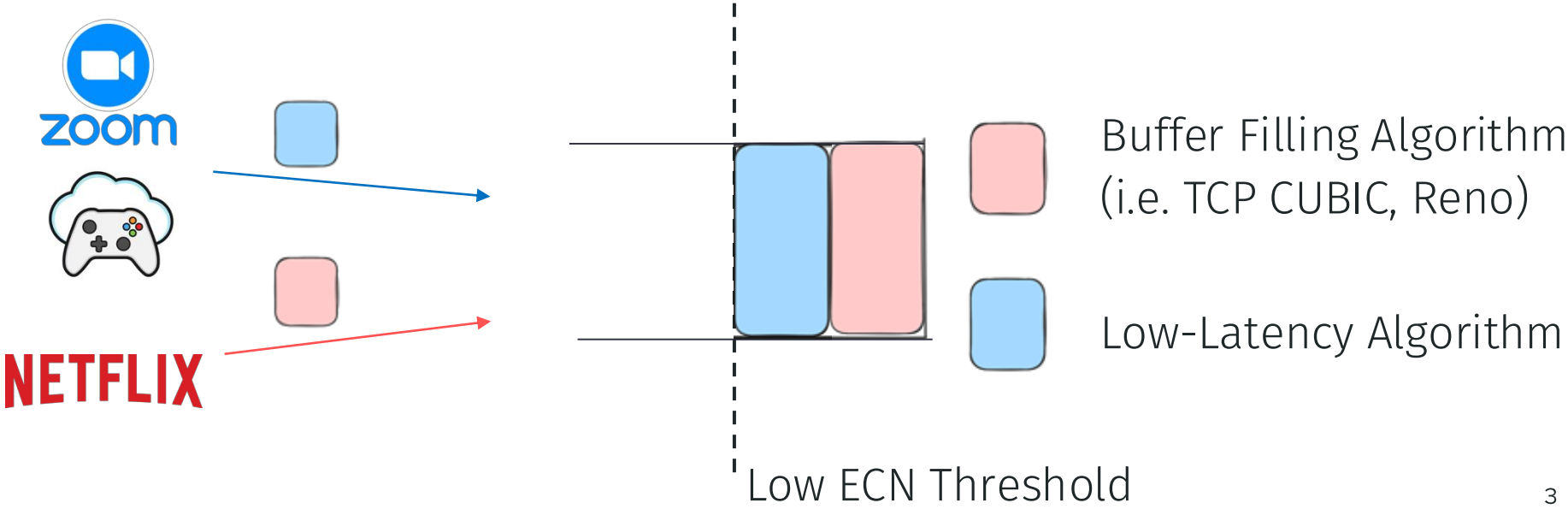
- Low-latency protocols suffer because they share a queue with buffer filling loss-based congestion controls.



Low-Latency Protocols Coexistence Problem

Throughput Problem

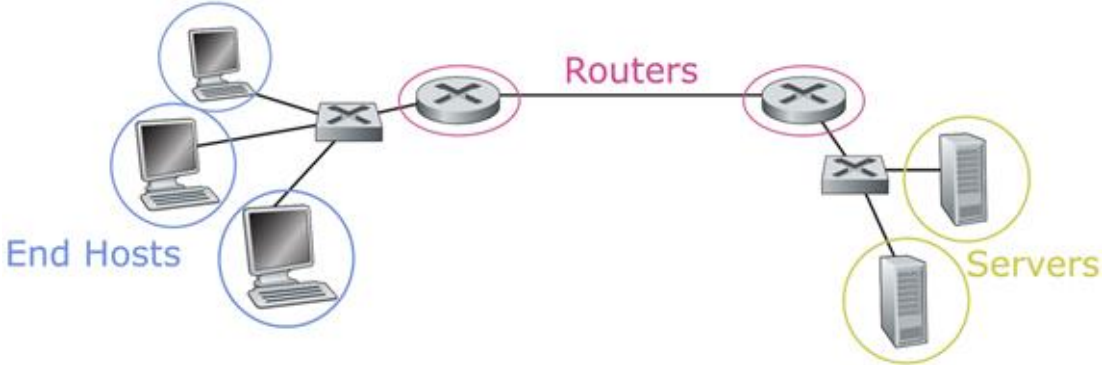
- Setting a low ECN threshold to help low-latency flows starves classic flows of bandwidth.



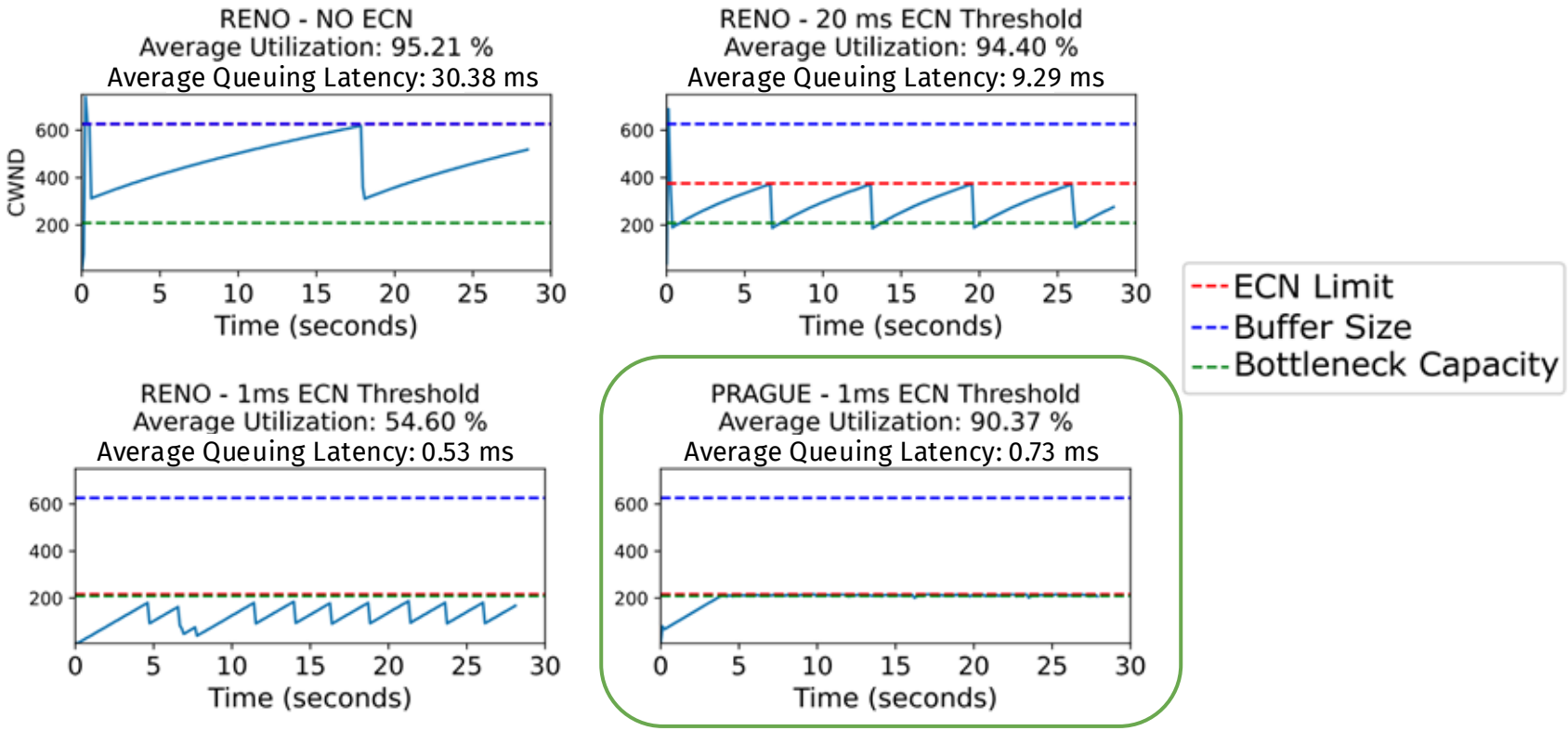
L4S: Low Latency, Low Loss and Scalable Throughput

Core Mechanisms:

- Scalable Congestion Control (Scalable CC)
- Accurate Explicit Congestion Notification (AccECN)
- Dual Queue Active Queue Management (DualQ AQM)



Scalable CC is necessary to maintain very low queuing delay



Dual Queue AQM

Classic and L4S flows do not co-exist well! We need to isolate them.

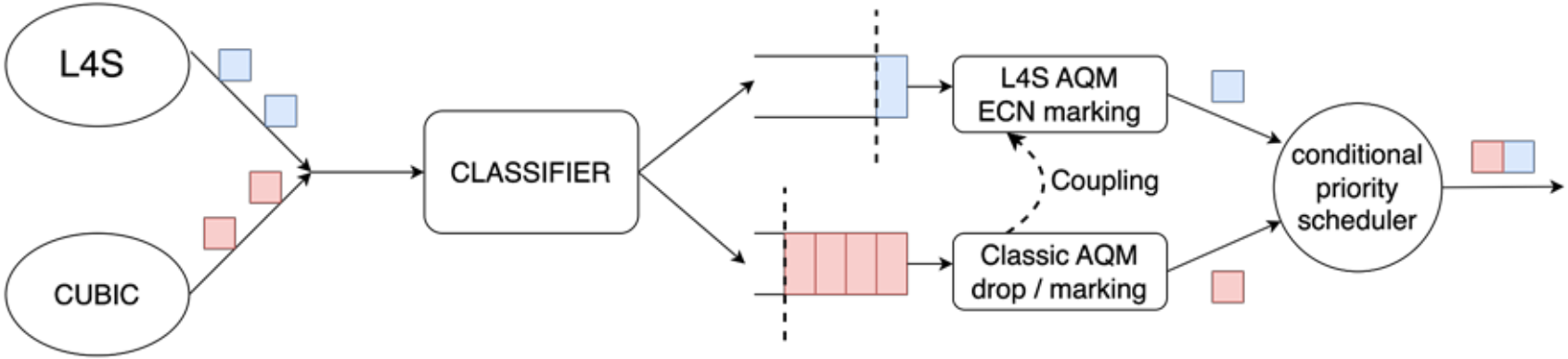


Fig: Dual Queue Coupled AQM

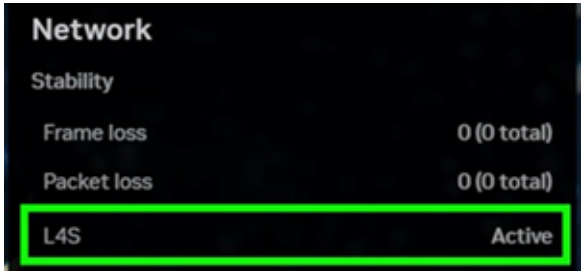
Binary Codepoint	Codepoint Name	Meaning
00	Not-ECT	Not ECN-capable transport
01	ECT(1)	L4S-capable transport
10	ECT(0)	Not L4S-capable transport
11	CE	Congestion Experienced

Table I: L4S codepoints and meaning

L4S is Gaining Industry Traction

Nokia

Apple



Nvidia

Without L4S

High latency peaks cause video freezes

Managed Latency with L4S

Consistent low latency for smooth AR experience

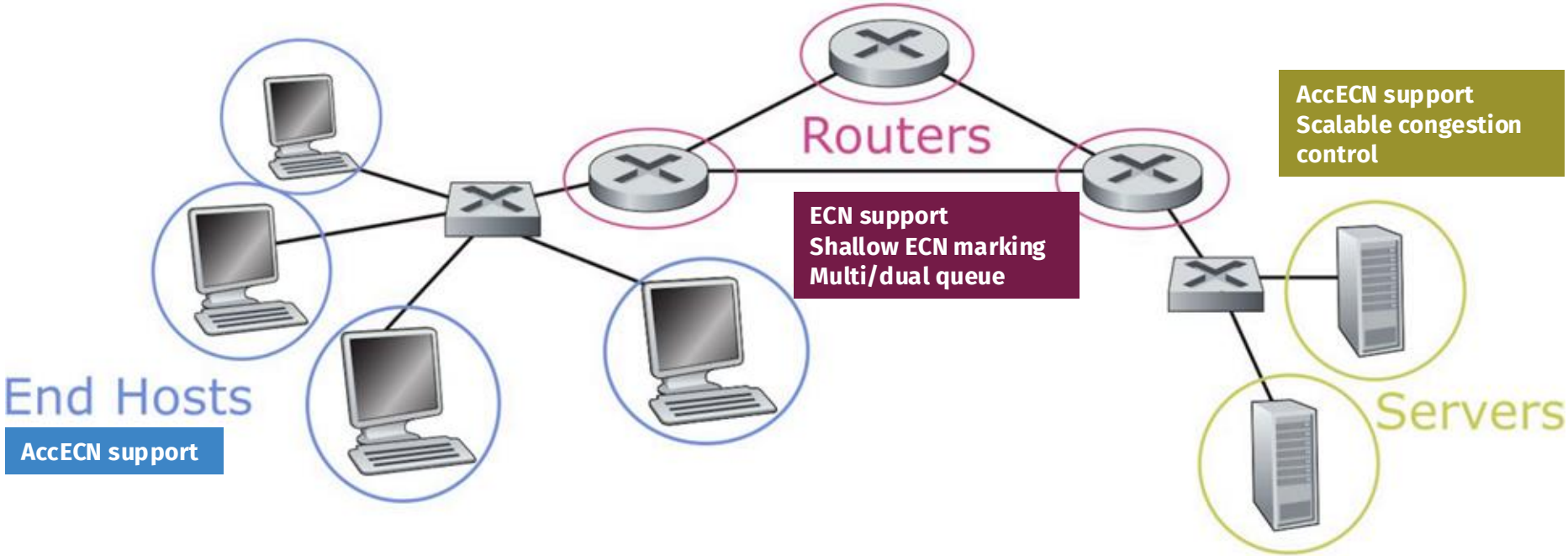
Ericsson

Comcast

T-Mobile Is First to Unlock L4S in Wireless — A Key Step Toward a Smarter, Programmable 5G

T-Mobile

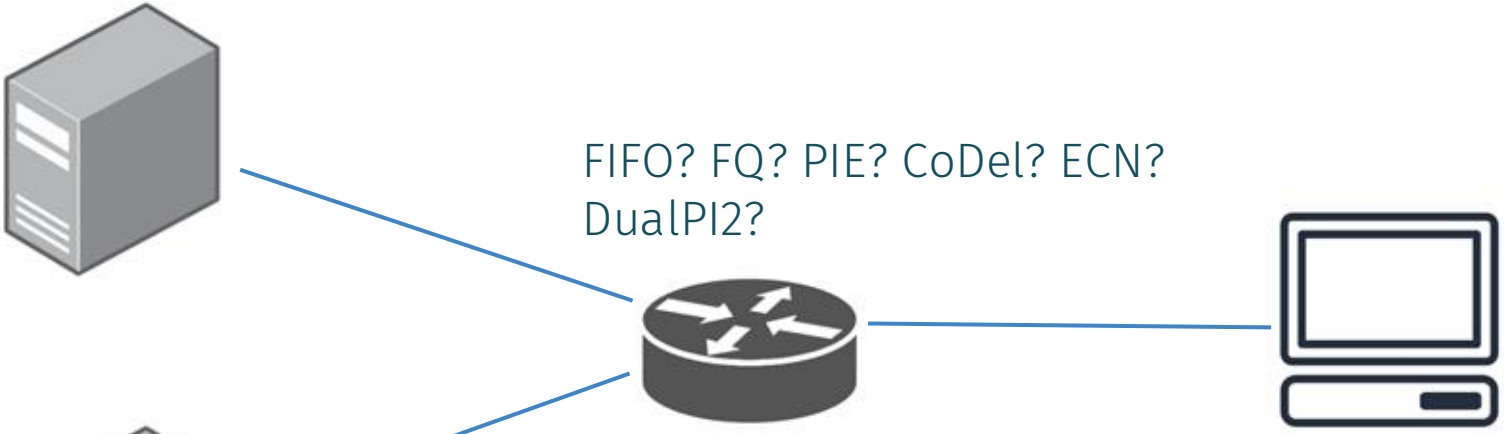
L4S involves both endpoints and routers!



A new protocol won't be deployed universally, all at once - there will be some incremental deployment.

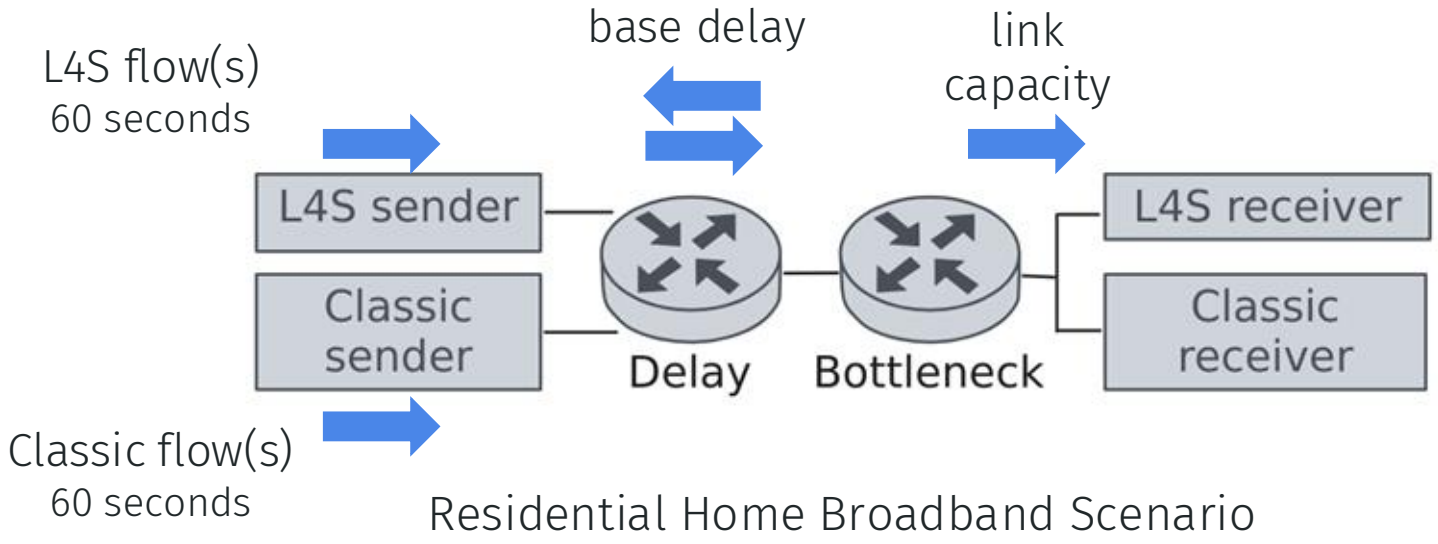
As a sender, should I use L4S?

Sender is not aware of the bottleneck type and competing flows!



Experiment design on FABRIC testbed: topology

A flow is most likely to encounter a bottleneck either at a peering point, or at the access link. We emulate network conditions that are representative of an access link: **10 ms base RTT, 100 Mbps bottleneck link capacity**.

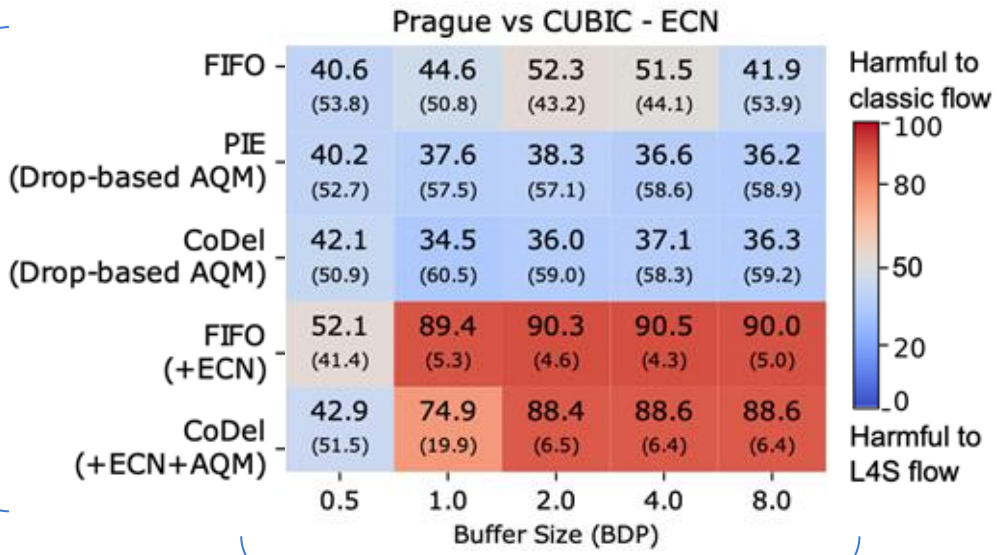


Residential Home Broadband Scenario

FABRIC

Example Results to Illustrate Our Methodology

Different bottleneck types



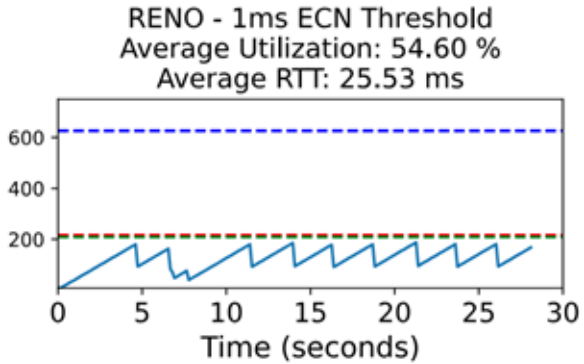
Throughput share

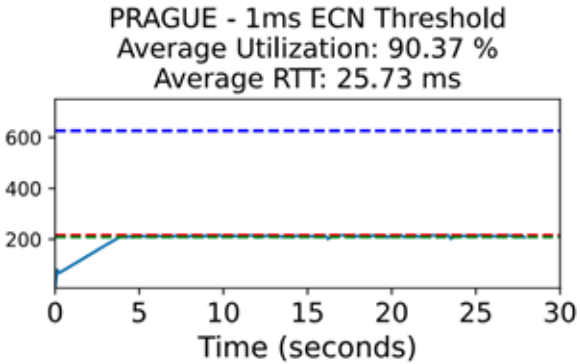
Different buffer sizes

Single Queue with ECN Support - L4S flow dominates!

- Single queue + ECN: L4S flow captures 75-90% of capacity (leaves classic flow with as little as 10%).

- But it is not prevalent on the Internet!





DualPI2 does not work well with BBR!

- DualPI2 (L4S-compatible queue): L4S flow gets less than 5-15% of capacity against BBR (v1/v2/v3) flow.

- This finding is concerning because of the (increasing) prevalence of BBR on the Internet.

Fair queuing allows safe coexistence

- With a fair queue that enforces flow isolation, any variety of L4S congestion control can coexist with any variety of classic congestion control.

- L4S and Classic flows share the capacity fairly.

- Only L4S-aware FQ-CoDel also supports ultra-low latency.

Low-latency requires full L4S support

- Low-latency benefits are realized only when all of the ingredients are in place (endpoints + middlebox).
- DualPI2 or L4S-aware FQ bottlenecks with L4S flow
- L4S flows achieve sub-millisecond queuing latency.

Internet-Scale L4S Deployment Is Not Yet Reliable

→ Early results from controlled deployments have been promising but...

→ Given that the sender of an L4S flow cannot be sure what type of queue is at the bottleneck router or what other flows will share it, safe coexistence across the Internet cannot be guaranteed right now.

→ A flow using L4S congestion control may be "starved" by or "starve" other flows at a shared bottleneck.

Summary

AQM	CUBIC	BBRv1	BBRv2 (one-sided DCTCP ECN)	BBRv2 (two-sided DCTCP ECN)	BBRv3
SQ (Drop)	✓ (Fig. 3, 15)	✗ (Fig. 13)	✗ Prague ✓ L4S-BBRv2 (Fig. 7, 11, 19, 25, 27)		
SQ (ECN)	✗ (Fig. 3, 15)	✗ (Fig. 13)	✗ (Fig. 7, 19)	✓ (Fig. 11, 19)	✗ (Fig. 25, 27)
DualPI2 (ϟ)	✓ (Fig. 4, 16)	✗ (Fig. 13)	✓ (Fig. 8, 20)	✗ (Fig. 11, 20)	✗ (Fig. 26, 28)
FQ (ϟ if L4S-aware)	✓ (Fig. 4, 13, 8, 26, 11, 16, 20, 28)				

Table 1: Can a sender turn on L4S (✓) or would it harm/be harmed if it did (✗)? (ϟ indicates settings where ultra-low latency is achieved, otherwise it is not.)

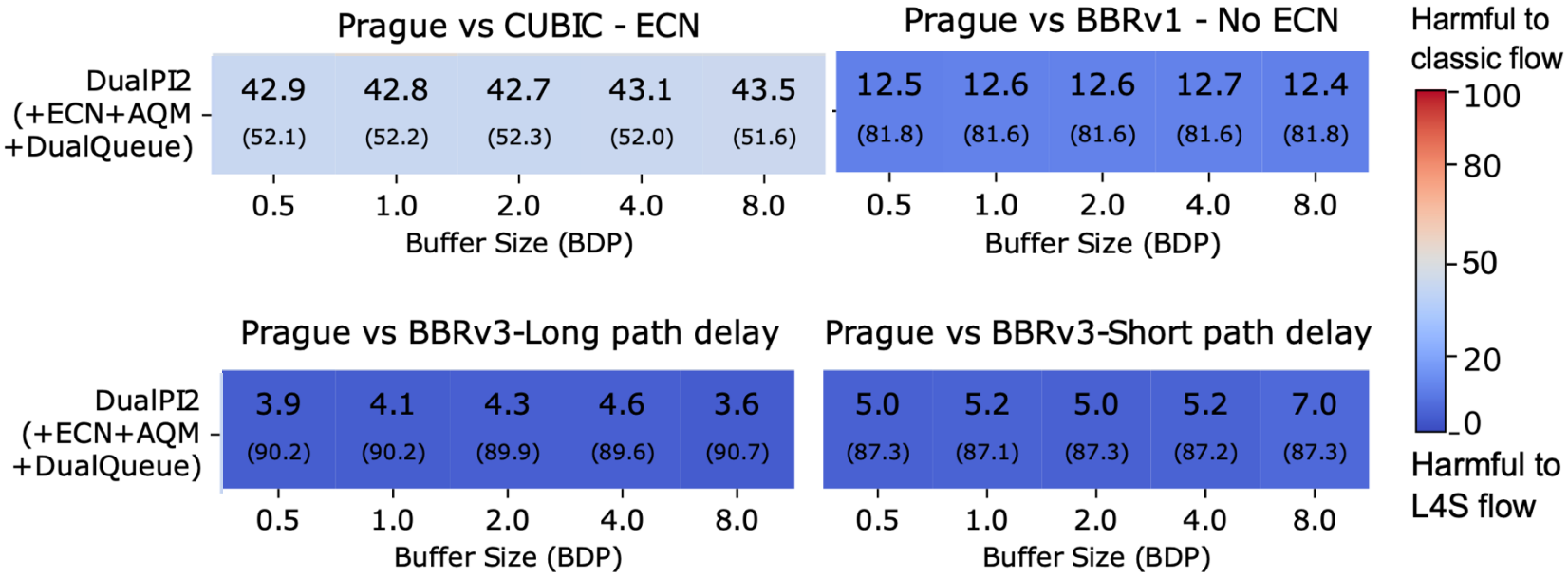
These results were published at:

- F. B. Sarpkaya, A. Srivastava, F. Fund, S. Panwar. "To switch or not to switch to TCP Prague? Incentives for adoption in a partial L4S deployment." **ANRW 2024**

- F. B. Sarpkaya, F. Fund, S. Panwar. "To adopt or not to adopt L4S-compatible congestion control? Understanding performance in a partial L4S deployment." **PAM 2025**

Our focus: BBR Starves L4S Traffic at DualPI2

Throughput share



Model-Based vs. Loss-Based Traffic Coexistence Problem

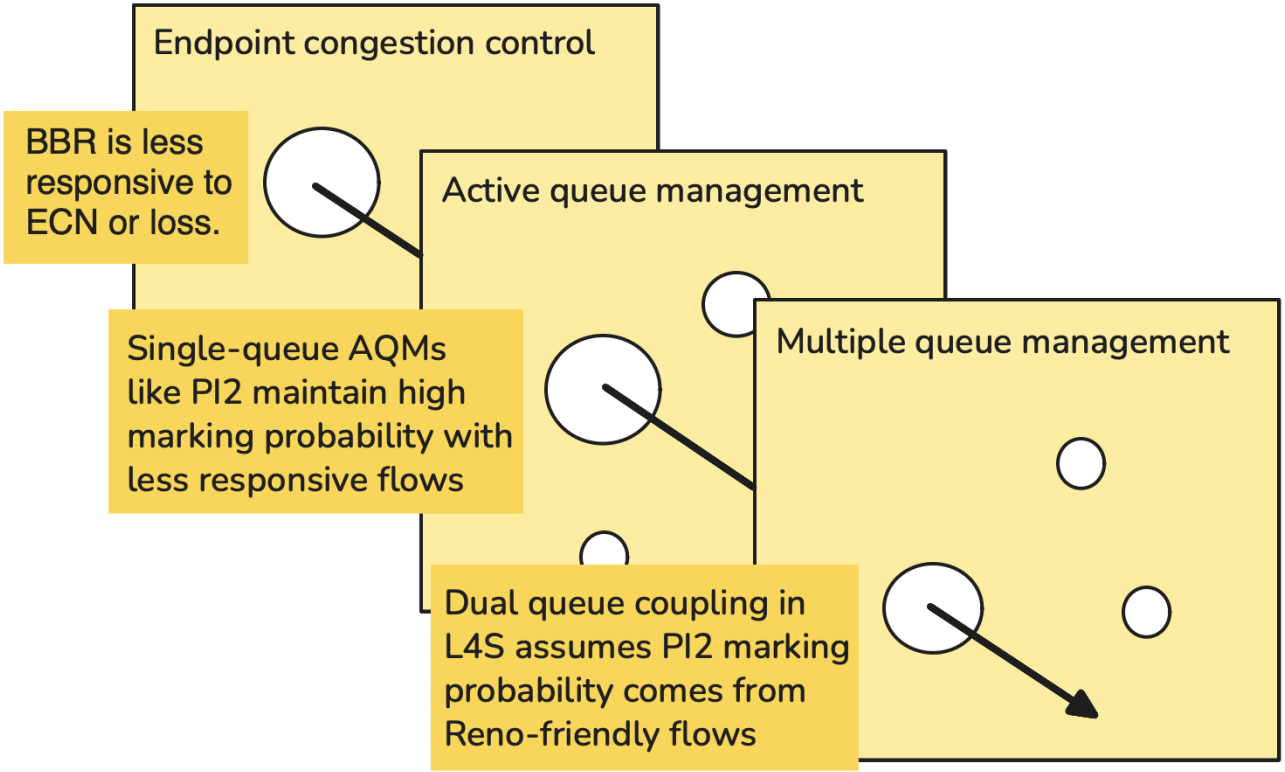
→ BBR is a model-based algorithm — it estimates bandwidth and RTT rather than reacting directly to loss or ECN marks

→ Under AQM control, BBR's weaker responsiveness forces the AQM to sustain a persistently high congestion signal

→ This suppresses co-existing loss-based flows — even in conventional single-queue AQMs

→ We will show that DualPI2 inherits this same fragility

Swiss Cheese Model



Motivating Experiments

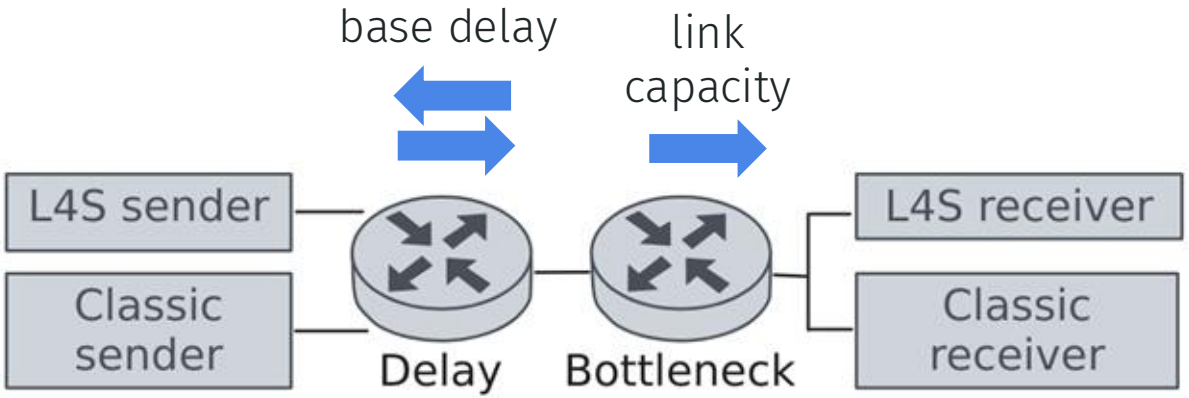
→ Instead of bulk flows, we emulate realistic traffic:

→ Video Streaming, Video Call, File Transfer

→ MGEN declarative traffic generator tool.

→ 100 Mbps bottleneck, 30 ms RTT, 8 BDP buffer, AQMs with default parameters

→ 5 application flows + 5 competing bulk flows



FABRIC

Application Workload Models

Application	Model	Metric
Video Streaming	- Chunk-based, 10 Mbps, 2s chunks - TCP CUBIC, TCP Prague - 60 seconds video, 120 seconds experiments	1. Total Rebuffering Duration 2. Fraction of Played Chunks Affected by Rebuffering 3. Fraction of Played Chunks
Video Calling	- Periodic UDP, 5 Mbps, 1200B packets - UDP Prague - 120 seconds experiments	1. Mean achieved rate fraction 2. Mean p95 RTT 3. Bad interval fraction (<90% target rate OR >150 ms RTT)
File Transfer	- Bulk, 100 MB - TCP CUBIC, TCP Prague	1. Flow Completion Time (FCT)

Single Queue AQM CUBIC Results – Video Streaming

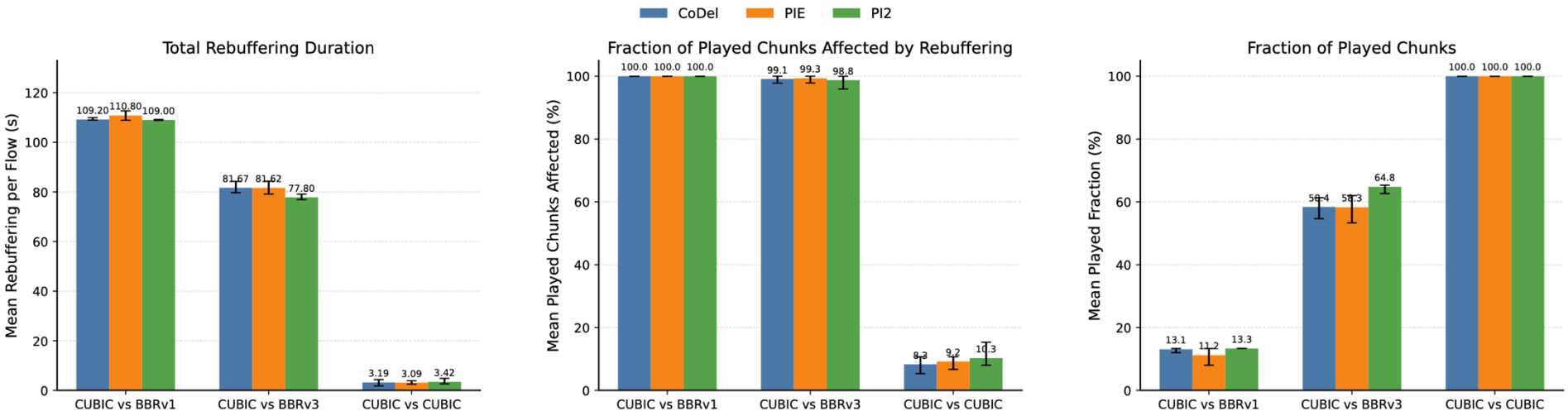


Figure: Video-on-demand streaming performance of CUBIC flows sharing a single queue bottleneck with BBR or CUBIC bulk traffic.

Single Queue AQM CUBIC Results – File Transfer

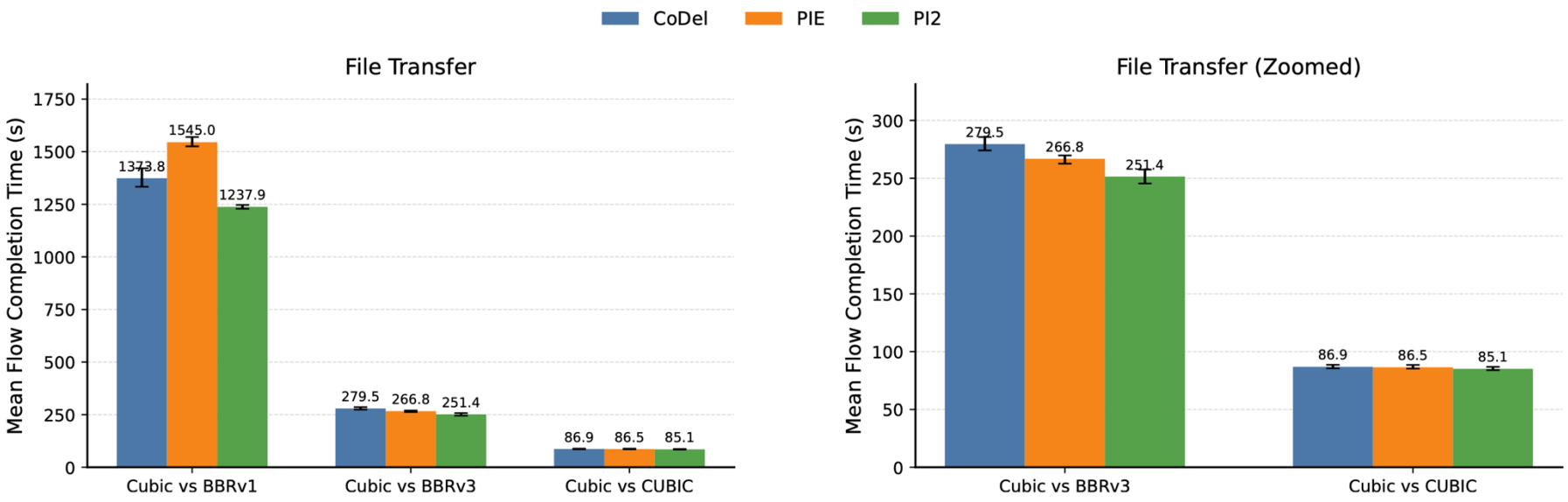


Figure: File transfer performance of CUBIC flows sharing a single-queue bottleneck with BBR or CUBIC bulk traffic.

Dual Queue AQM L4S Results – Video Streaming

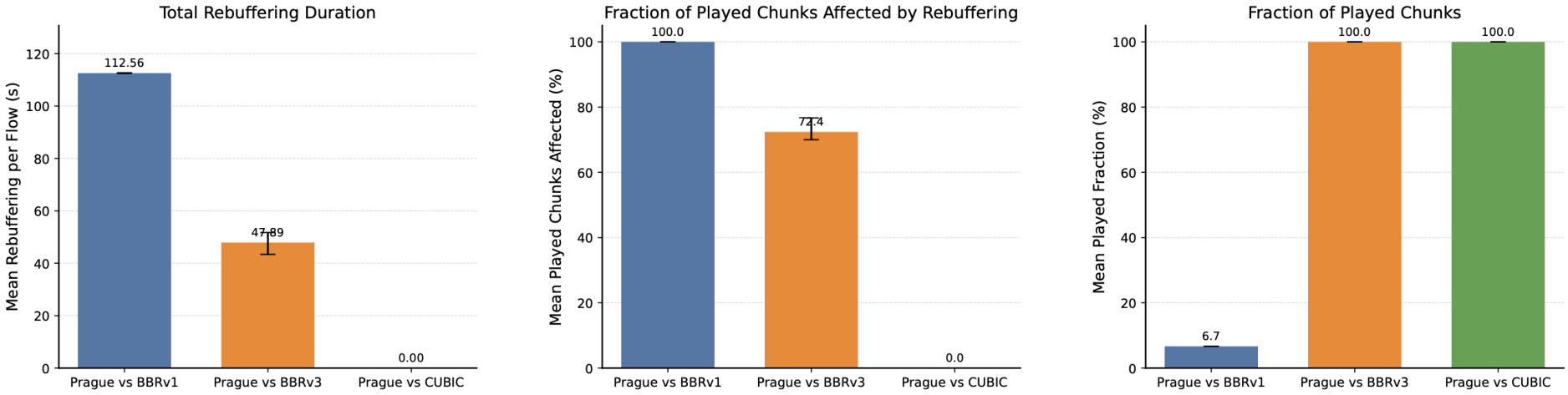


Figure: Video-on-demand streaming performance of Prague flows sharing a DualPI2 bottleneck with BBR or CUBIC bulk traffic.

Dual Queue AQM L4S Results – Video Call

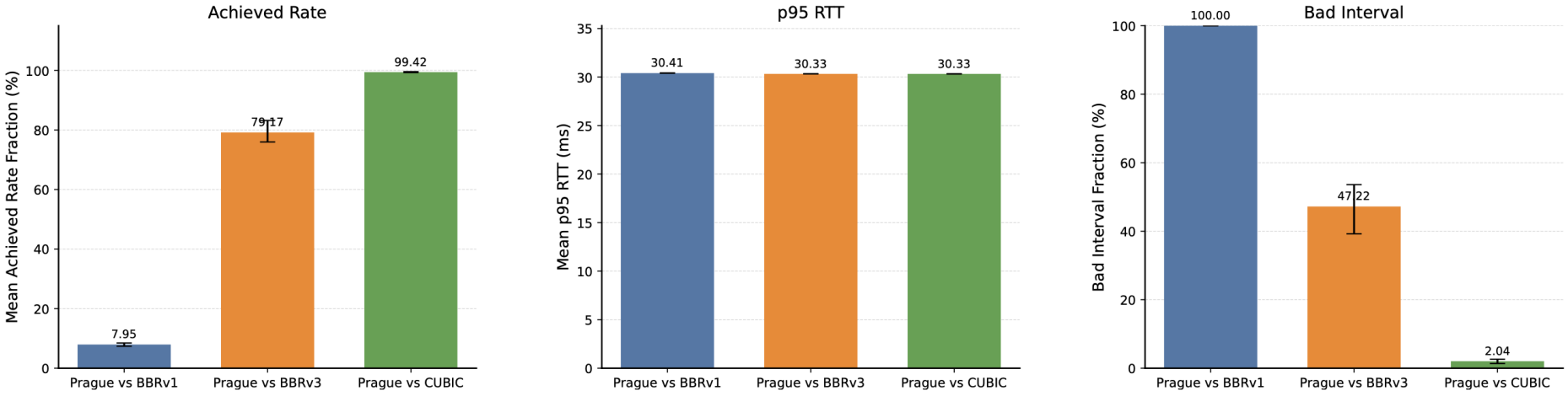
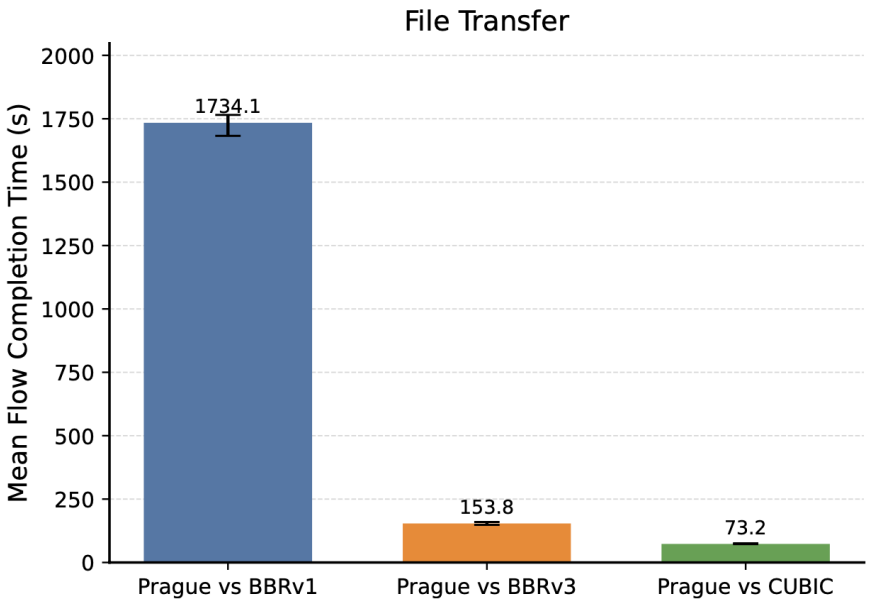


Figure: Video-call performance of Prague flows sharing a DualPI2 bottleneck with BBR or CUBIC bulk traffic.

Dual Queue AQM L4S Results – File Transfer



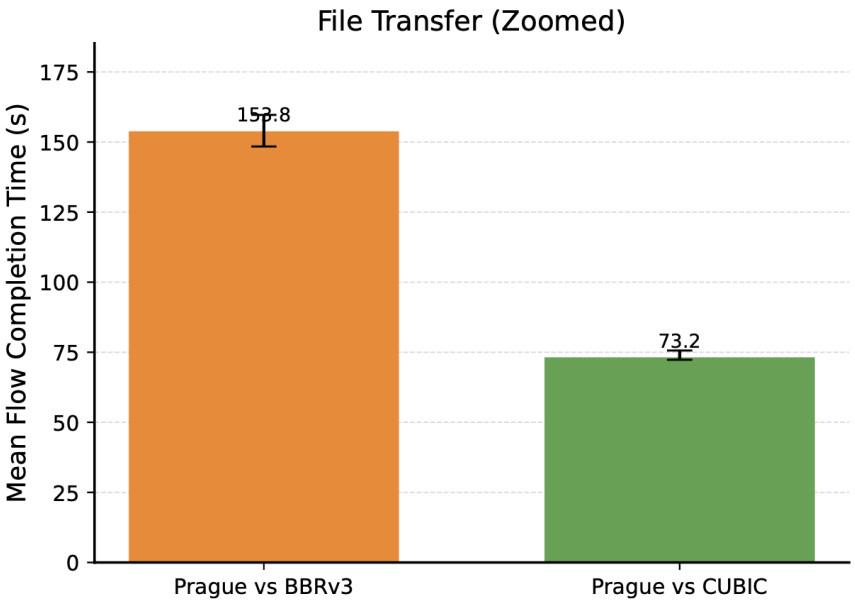


Figure: File transfer performance of Prague flows sharing a DualPI2 bottleneck with BBR or CUBIC bulk traffic.

Takeaways from Motivating Experiments

→ Coexistence problems between model-based and loss-based traffic are visible at the application level.

→ These problems already appear under conventional single queue AQMs, where BBR bulk traffic can substantially degrade the performance of competing applications.

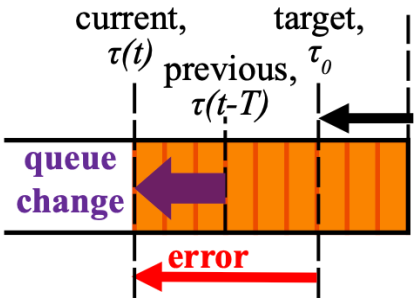
→ DualPI2 inherits this fragility in the L4S setting.

Insight: BBR causes high congestion signal on AQMs

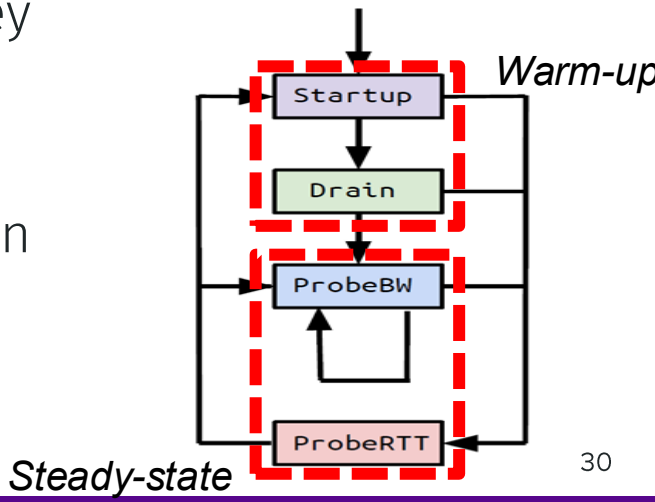
→ PI2 is a feedback controller — it raises marking probability p' until queue delay reaches its target

→ Loss-based algorithms work well because they are responsive to these signals.

→ BBR responds weakly to p' → PI2 must sustain high p' just to hold the queue near target



$$\Delta p = \alpha(\text{error}) + \beta(\text{queue change})$$



Root cause Analysis: Fluid Model Framework

→ We build on the BBRv1/v2 fluid model by Scherrer et al. (IMC'22)

→ We extend it in two ways:

- Add PI2 as the bottleneck AQM (they only had FIFO and RED)
- Extend the sender model from BBRv2 to BBRv3

→ Validated against Linux experiments on FABRIC

Link-Arrival Rate.

The arrival rate $y_\ell(t)$ at link ℓ is

$$y_\ell = \sum_{i \in U_\ell} x_i\left(t - d^{\mathrm{f}}_{i,\ell}\right),$$

Queue Length.

$$\dot{q}_\ell = (1 - p_\ell) \cdot y_\ell - C_\ell, \quad q_\ell(t) \in [0, B_\ell],$$

Latency.

$$\tau_{\pi_i} = \sum_{\ell \in \pi_i} \tau_\ell = \sum_{\ell \in \pi_i} d_\ell + \frac{q_\ell}{C_\ell}.$$

Loss Probability

$$p_\ell(t) = \sigma\left(y_\ell(t) - C_\ell\right) \cdot \left(1 - \frac{C_\ell}{y_\ell}\right) \cdot \left(\frac{q_\ell}{B_\ell}\right)^L$$

ProbeRTT

BBRv2 Fluid Model:

$$T_i^{\mathrm{prt}}\big|_{\mathrm{v2}} = m_i^{\mathrm{prt}} \cdot 0.2 + (1 - m_i^{\mathrm{prt}}) \cdot 10$$

BBRv3 Extension:

$$T_i^{\mathrm{prt}}\big|_{\mathrm{v3}} = m_i^{\mathrm{prt}} \cdot 0.2 + (1 - m_i^{\mathrm{prt}}) \cdot 5$$

Pacing Rate

BBRv2 Fluid Model:

$$x_i^{\mathrm{pcg}}\big|_{\mathrm{v2}} = x_i^{\mathrm{btl}} \cdot \left(1 + \frac{1}{4} \cdot \sigma\big(t_i^{\mathrm{pbw}} - \tau_i^{\min}\big) \cdot (1 - m_i^{\mathrm{dwn}}) - \frac{1}{4} \cdot m_i^{\mathrm{dwn}}\right)$$

BBRv3 Extension:

$$x_i^{\mathrm{pcg}}\big|_{\mathrm{v3}} = 0.99 \cdot x_i^{\mathrm{eff}} \cdot \left(1 + \frac{1}{4} \cdot \sigma\big(t_i^{\mathrm{pbw}} - \tau_i^{\min}\big) \cdot (1 - m_i^{\mathrm{dwn}}) - \frac{1}{10} \cdot m_i^{\mathrm{dwn}}\right)$$

$$x_i^{\mathrm{eff}} = \min\big(x_i^{\mathrm{btl}},\ x_i^{\mathrm{lo}}\big)$$

Congestion Window Gain

BBRv2 Fluid Model:

$$w_i^{\mathrm{pbw}}\big|_{\mathrm{v2}} = \min\Big(2 \cdot \bar{w}_i,\ \ (1 - m_i^{\mathrm{crs}}) \cdot w_i^{\mathrm{hi}} + m_i^{\mathrm{crs}} \cdot w_i^{\mathrm{lo}}\Big)$$

BBRv3 Extension:

$$w_i^{\mathrm{pbw}}\big|_{\mathrm{v3}} = \min\Big(g_i^{\mathrm{cwnd}} \cdot \bar{w}_i,\ \ \min\big(\hat{w}_i^{\mathrm{lo}},\ (1 - 0.15\, m_i^{\mathrm{crs}}) \cdot w_i^{\mathrm{hi}}\big)\Big)$$

$$g_i^{\mathrm{cwnd}}\big|_{\mathrm{v3}} = 2 + \frac{1}{4} \cdot \phi_i^{\mathrm{up}}$$

$$\hat{w}_i^{\mathrm{lo}} = \underbrace{(1 - m_i^{\mathrm{crs}})(1 - m_i^{\mathrm{dwn}})}_{=1 \text{ during Refill/Up}} \cdot W_{\mathrm{big}} + \big(1 - (1 - m_i^{\mathrm{crs}})(1 - m_i^{\mathrm{dwn}})\big) \cdot w_i^{\mathrm{lo}}$$

New State Variable – Bandwidth Lower Bound

$$x_i^{\text{eff}} = \min(x_i^{\text{btl}},\ x_i^{\text{lo}})$$

- It is a short-term estimate of the safe sending bandwidth.
- It is effectively infinite during Refill and Up.
- During Cruise and Down, if loss exceeds 2% and BBR is not probing, it decays by 30% once per RTT.

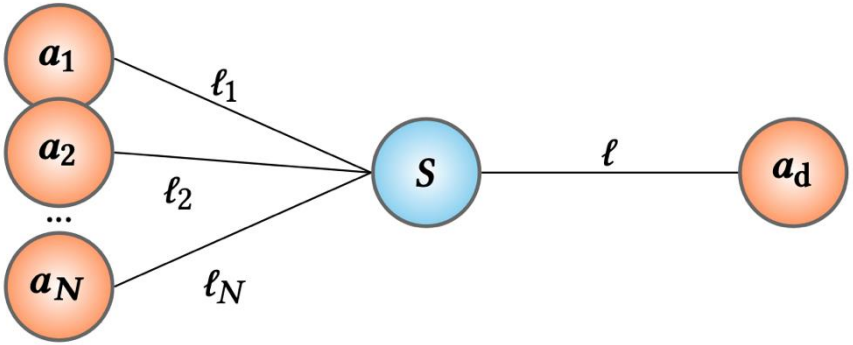
Validation

→ Senders, N=10

→ 100 Mbps Bottleneck Rate

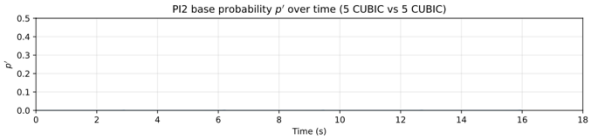
→ 30ms RTT

→ 5 CUBIC vs 5 CUBIC
→ 5 CUBIC vs 5 BBRv1
→ 5 CUBIC vs 5 BBRv3

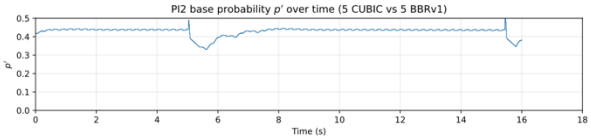


→ We trace the congestion signal on the bottleneck.

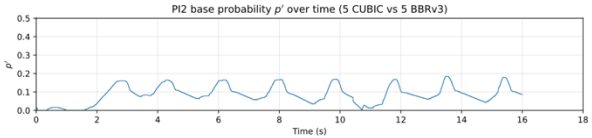
Validation



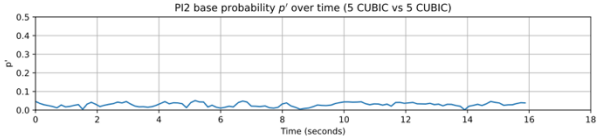
(a) Fluid model: CUBIC vs. CUBIC



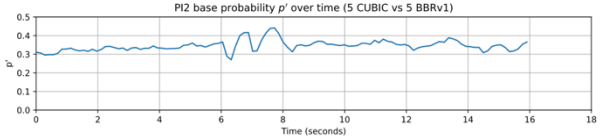
(b) Fluid model: CUBIC vs. BBRv1



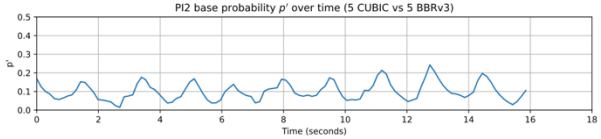
(c) Fluid model: CUBIC vs. BBRv3



(d) Linux: CUBIC vs. CUBIC

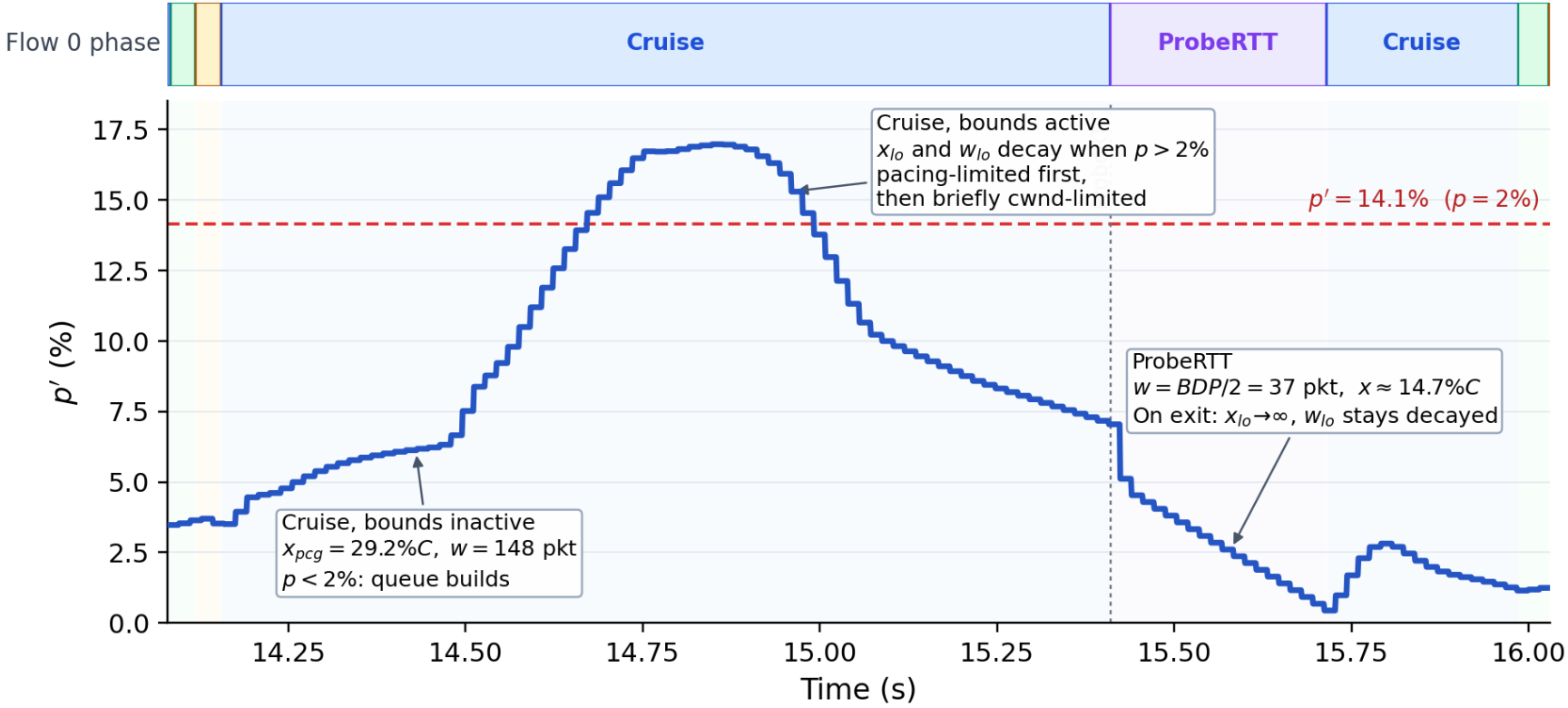


(e) Linux: CUBIC vs. BBRv1

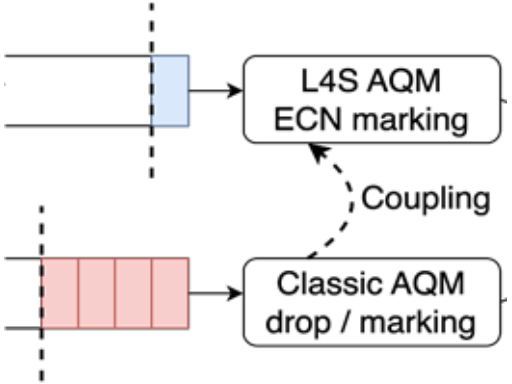


(f) Linux: CUBIC vs. BBRv3

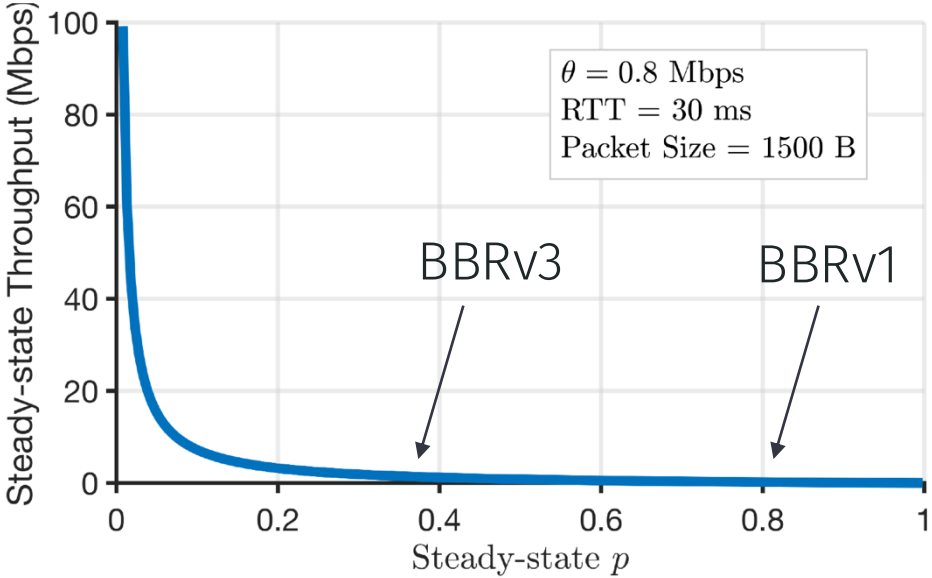
5 BBRv3 vs 5 CUBIC on PI2



DualPI2 Coupling Transfers the Problem to L4S



k=p'



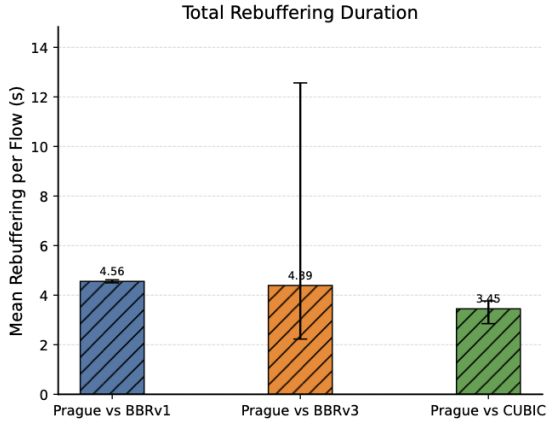
$$R_P(\hat{p}) = \theta_0 \cdot \frac{(1-\hat{p})}{\hat{p}}$$

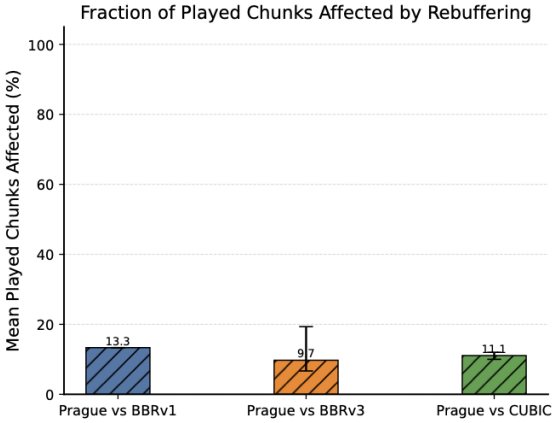
Mitigation Directions

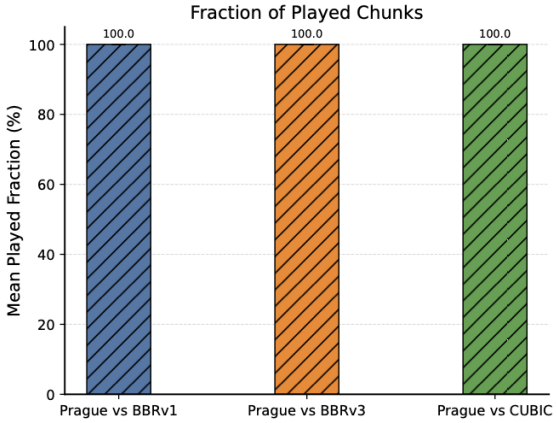
→ Flow Queuing (i.e FQ-CoDel with L4S support)

→ Capped coupling in DualPI2 (our proposed explatory mechanism)

FQ-CoDel as Practical Baseline







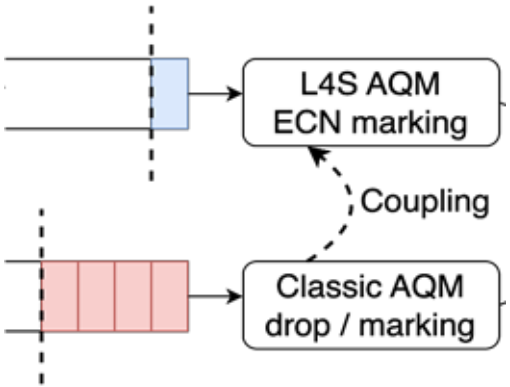
Capped Coupling Mechanism

→ Root cause: persistently large p’ in Classic queue transfers to the L4S queue.

→ Fix: cap the coupled p' seen by the L4S queue at a value derived from a target per-flow Prague throughput.

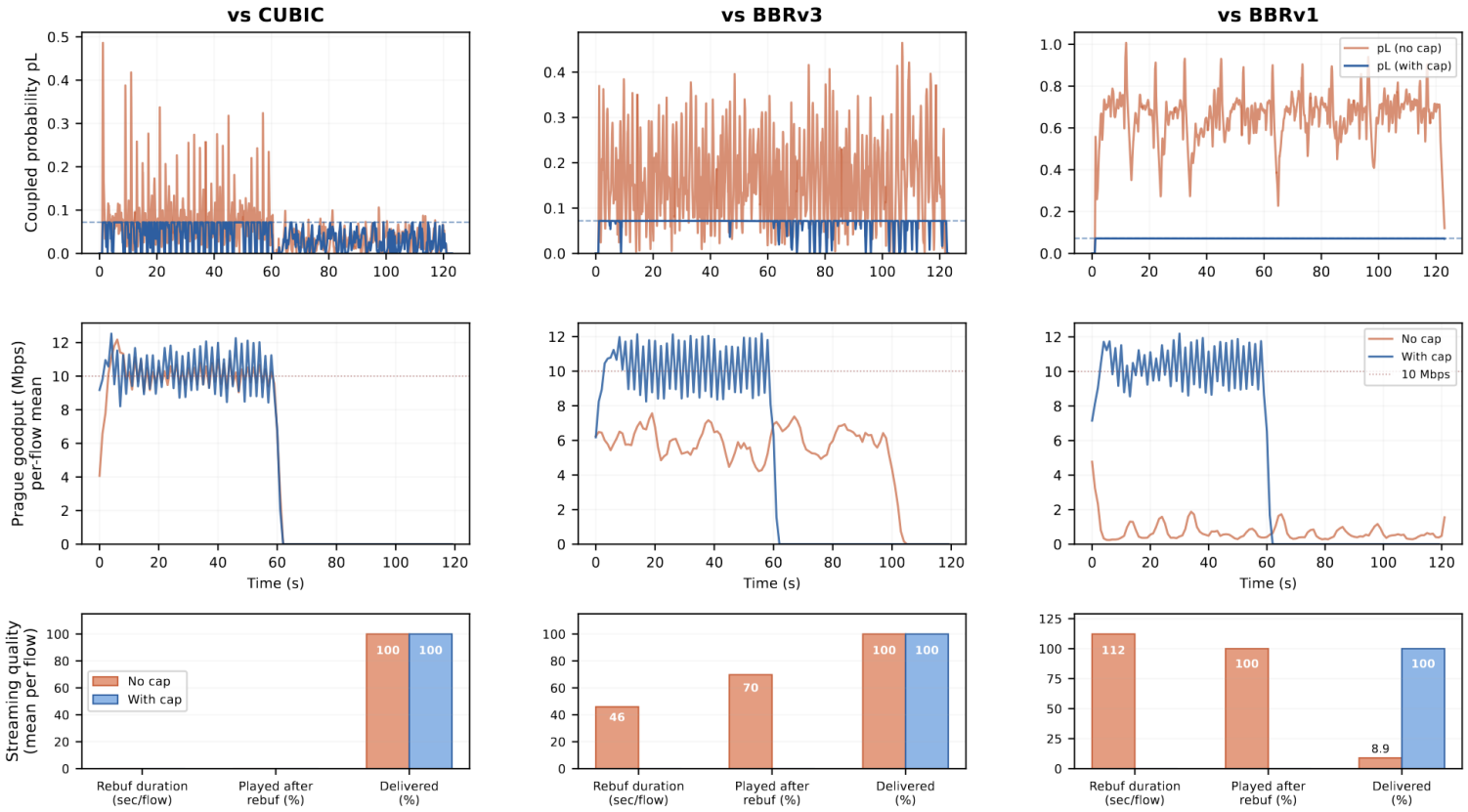
→ given target throughput, and RTT, compute p̂_cap from the Prague throughput model

→ target = 10 Mbps per flow, RTT = 30 ms → cap = 0.072



$$R_P(\hat{p}) = \theta_0 \cdot \frac{(1-\hat{p})}{\hat{p}}$$

Capped Coupling Mechanism



Limitations of Capped Coupling

→ Router cannot observe Prague RTT directly — must be configured as a parameter

→ No per-flow state — cannot account for number of active Prague flows

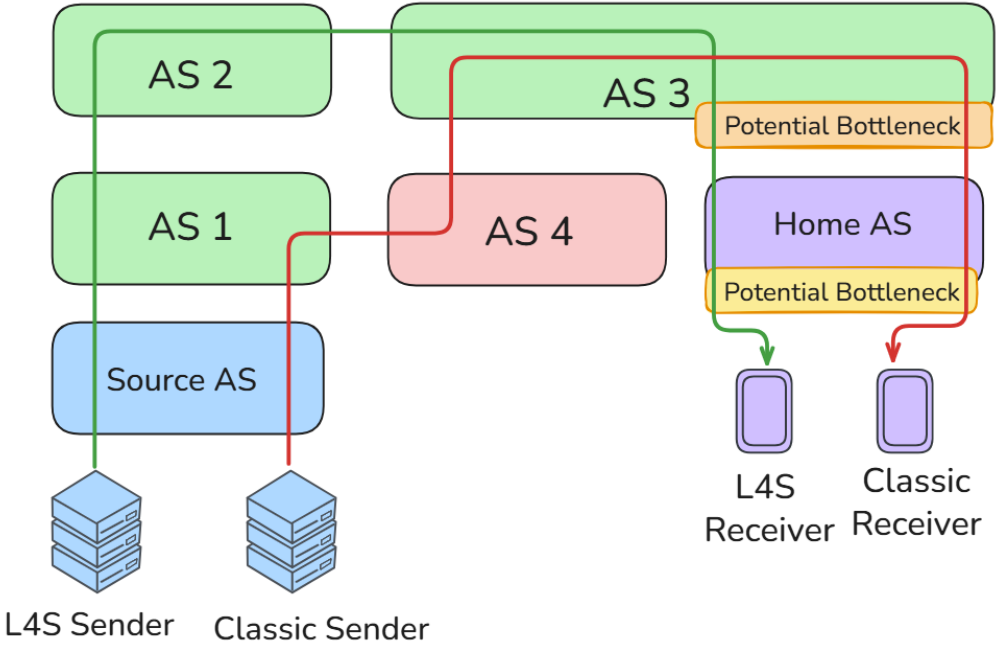
→ Transient Classic queue delay spikes can be larger when cap prevents Prague from backing off

→ Capped coupling is an exploratory direction, not a complete solution. FQ remains the strongest practical mitigation.

Future Work

Extending Low Latency Service Across the Internet

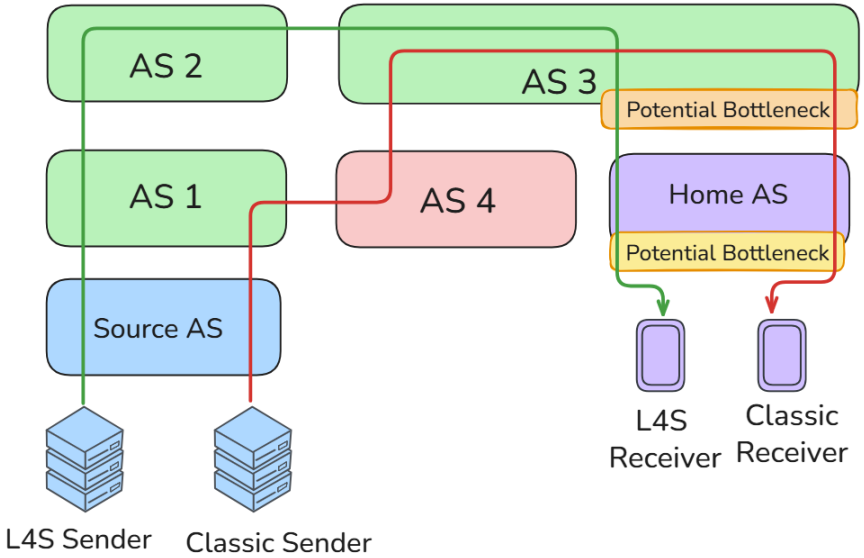
- Use SRv6 to steer low-latency flows through L4S-compatible paths within an AS, and BGP communities to signal compatible ingress points between ASes.
- DSCP marking at ingress → SRv6 path selection within AS → priority queueing at peering points → DualPI2 at receiver access.



Economic Framework for L4S Deployment Decisions

$$U(\lambda, D) = \gamma \cdot \mathrm{QoS}_{\mathrm{LL}}(\lambda, D) - C(D)$$

- D which segments are upgraded (subset of {access, peering, transit, …})
- $\mathrm{QoS}_{\mathrm{LL}}(\lambda, D)$ measured low-latency performance, given demand λ and deployment D
- $C(D)$ is the deployment cost
- γ is the revenue per unit of QoS



- Solving for demand thresholds λ^* identifies which deployment paths are economically viable.

Publications

- **F. B. Sarpkaya**, A. Srivastava, F. Fund, S. Panwar. "To switch or not to switch to TCP Prague? Incentives for adoption in a partial L4S deployment." **ANRW 2024**
- **F. B. Sarpkaya**, F. Fund, S. Panwar. "To adopt or not to adopt L4S-compatible congestion control? Understanding performance in a partial L4S deployment." **PAM 2025**
- **F. B. Sarpkaya**, A. Srivastava, F. Fund, S. Panwar. "BBR's Sharing Behavior with CUBIC and Reno." **IFIP Networking 2025**
- **F. B. Sarpkaya**, A. Francini, B. Erman, S. Panwar. "Evaluation of TCP Congestion Control for Public High-Performance Wide-Area Networks." **HPSR 2026**
- M. Ying, **F. B. Sarpkaya**, S. Bakirtas, E. Erkip, T. S. Rappaport, S. Rangan. "Capacity of a Binary Channel with a Time-Bounded Adversary." **Asilomar 2024**

Conclusions

→ L4S senders cannot be assured of safe coexistence when the bottleneck type and competing traffic are unknown.

→ Poor coexistence between model-based and loss-based traffic under AQM control is a general problem — DualPI2 inherits it through its coupling mechanism.

→ FQ-CoDel avoids the problem through per-flow isolation; capped coupling in DualPI2 improves robustness when per-flow isolation is not feasible.

Thank You for Listening !